

Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models ¹
Student Name	NavinKumar S
Register Number	3122247001039
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/26

1 Objective

To implement Linear, Ridge, Lasso, and Elastic Net regression models to predict a continuous target variable – the requested loan amount – tune regularization hyperparameters via GridSearchCV, evaluate performance using multiple regression metrics, visualize model behaviour, and analyze overfitting, underfitting, and bias–variance characteristics.

2 Problem Statement

Given the Kaggle *Predict Loan Amount Data* dataset of 20,000 customer loan applications with demographic, financial, employment, and property-related features, the problem is regression: predict the continuous **Loan Amount Request** (USD) for each applicant. The input is a mixed categorical/numerical feature vector per applicant; the output is a continuous dollar value. The objective is to determine which regularization strategy (if any) generalizes best, and to characterize the bias–variance behaviour of each model on this dataset.

3 Dataset Description

Dataset Name	Predict Loan Amount Data
Dataset Source	Kaggle – <i>Predict Loan Amount Data</i>
Number of Samples	20,000
Number of Features	19 used (10 categorical, 9 numerical) – expands to 45 after one-hot encoding
Number of Classes	N/A – regression target (continuous), range 6,185–576,336 USD
Missing Values	Gender (31), Income (750), Income Stability (813), Type of Employment (4,689), Current Loan Expenses (83), Dependents (1,142), Credit Score (743), Has Active Credit Card (1,076), Property Age (892), Property Location (160), plus disguised “?” placeholders in Property Price (168) and Co-Applicant (77)
Train-Test Split	80:20 (16,000 train / 4,000 test), <code>random_state=42</code>

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution (not applicable – regression target)
- Correlation matrix (feature-vs-target correlations reported below)
- Feature distribution

Target Distribution

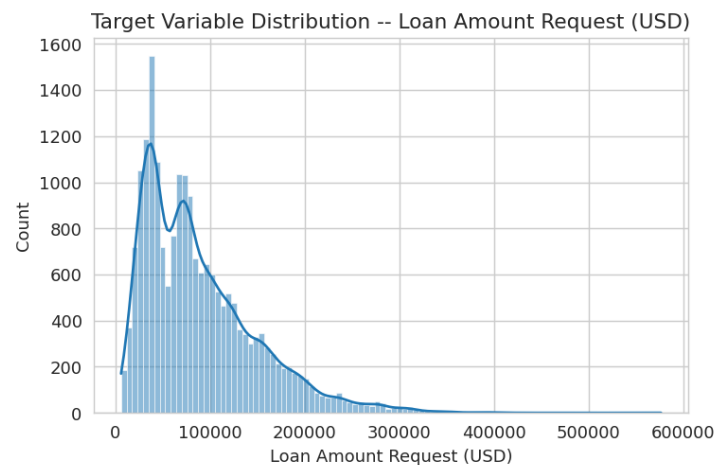


Figure 1: Target variable (Loan Amount Request) distribution

Inference: The requested loan amount is right-skewed (skewness ≈ 1.28), with most requests concentrated below 150,000 USD and a tail extending to over 500,000 USD. With 20,000 samples, the extreme tail has limited influence on the overall shape compared to a smaller dataset.

Feature Distribution / Correlation with Target

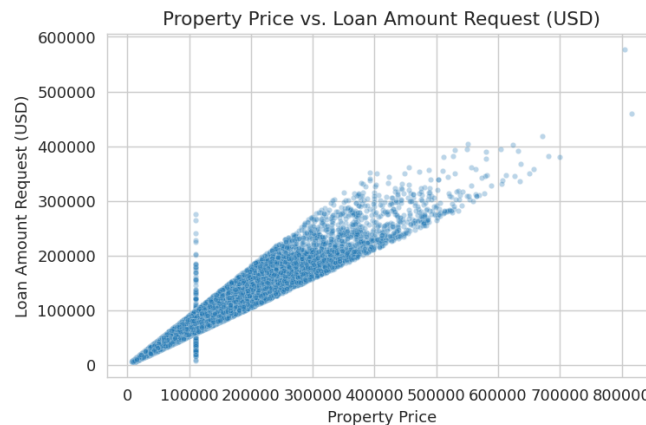


Figure 2: Property Price vs. Loan Amount Request

Inference: Property Price shows an extremely strong, almost linear positive relationship with the requested loan amount ($r \approx 0.961$) – by far the strongest relationship of any feature, and the dominant driver of every model’s predictions.

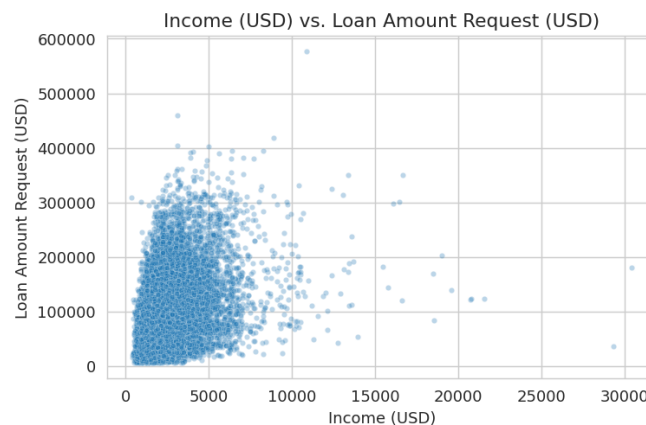


Figure 3: Income (USD) vs. Loan Amount Request

Inference: Income shows a much weaker positive relationship with loan amount ($r \approx 0.364$) than Property Price – applicants with similar incomes still request widely varying loan amounts, indicating income alone is a poor predictor without the property-value context.

5 Data Preprocessing

Explain:

- **Missing value handling:** Property Price and Co-Applicant encode missing data as the literal string “?” rather than a blank cell; both were converted to NaN and then to numeric type. All categorical columns with missing values were then imputed with their mode, and all numerical columns with missing values were

imputed with their median. **Customer ID**, **Name**, and **Property ID** were dropped as unique identifiers with no generalizable signal.

- **Encoding:** All ten categorical features (Gender, Income Stability, Profession, Type of Employment, Location, Expense Type 1, Expense Type 2, Has Active Credit Card, Property Type, Property Location) were one-hot encoded (`drop="first"`), expanding 19 raw input features into 45 encoded features.
- **Feature scaling:** Age, Income, Current Loan Expenses, Dependents, Credit Score, No. of Defaults, Property Age, Co-Applicant, and Property Price were standardized (zero mean, unit variance) using `StandardScaler`, fit only on the training split.
- **Train-test split:** 80:20 (16,000 train / 4,000 test rows), implemented inside a single `Pipeline` (preprocessing $\rightarrow$ model) to prevent data leakage across cross-validation folds and the held-out test set.
- **Feature engineering:** Not applied in this experiment – the 19 raw features were used directly after encoding/scaling.

6 Machine Learning Algorithm

Explain:

- **Working principle:** Linear Regression fits a linear combination of input features to minimize squared error against the continuous target. Ridge Regression adds an L_2 penalty on coefficient magnitudes to shrink them toward zero without eliminating any. Lasso Regression adds an L_1 penalty that can shrink coefficients exactly to zero, performing implicit feature selection. Elastic Net linearly combines both L_1 and L_2 penalties, controlled by a mixing parameter (`l1_ratio`).
- **Mathematical formulation:**

$$\hat{y} = w^\top x + b, \quad J(w) = \underbrace{\|y - Xw\|_2^2}_{\text{OLS loss}} + \underbrace{\alpha\|w\|_2^2}_{\text{Ridge}} \text{ or } \underbrace{\alpha\|w\|_1}_{\text{Lasso}} \text{ or } \underbrace{\alpha\left(r\|w\|_1 + \frac{1-r}{2}\|w\|_2^2\right)}_{\text{Elastic Net}}$$

where α controls overall regularization strength and r (`l1_ratio`) controls the L_1/L_2 mix in Elastic Net.

- **Advantages:** Linear Regression is simple, fast, and fully interpretable. Ridge handles multicollinearity gracefully by shrinking correlated coefficients together. Lasso performs automatic feature selection, useful when many features are irrelevant. Elastic Net balances both behaviours when the number/correlation structure of useful features is uncertain.
- **Limitations:** All four are linear models and cannot capture non-linear feature–target relationships. Ridge never produces a sparse (zero-coefficient) model even when many features are irrelevant. Lasso can behave erratically with strongly correlated features (arbitrarily picking one over another). Elastic Net introduces a second hyperparameter to tune, increasing search cost.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

8 Hyperparameter Selection

Parameter	Value
Ridge – best α (GridSearchCV)	10
Ridge – best CV R^2	0.9302
Lasso – best α (GridSearchCV)	10
Lasso – best CV R^2	0.9303
Elastic Net – best α (GridSearchCV)	0.01
Elastic Net – best l1_ratio (GridSearchCV)	0.8
Elastic Net – best CV R^2	0.9302

Note: Search spaces followed the assignment specification: Ridge $\alpha \in \{0.01, 0.1, 1, 10, 100\}$; Lasso $\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$; Elastic Net $\alpha \in \{0.01, 0.1, 1, 10\}$, l1_ratio $\in \{0.2, 0.5, 0.8\}$, all searched via 5-fold `GridSearchCV` optimizing R^2 .

9 Results

Metric (Regression)	Value (Best Model: Lasso Regression, Test Set)
MAE	10,890.31
MSE	232,800,718
RMSE	15,257.81
R^2	0.9356
Training Time (s)	0.0928

Note: The template's classification metrics (Accuracy/Precision/Recall/ROC-AUC) do not apply to this regression task and have been replaced with the standard regression metrics specified in the assignment (MAE, MSE, RMSE, R^2 , Training Time). All four models performed within 0.01% R^2 of each other on the test set (Linear 0.9355, Ridge 0.9355, Lasso 0.9356, Elastic Net 0.9356).

10 Result Visualization

Include all graphs generated during the experiment.

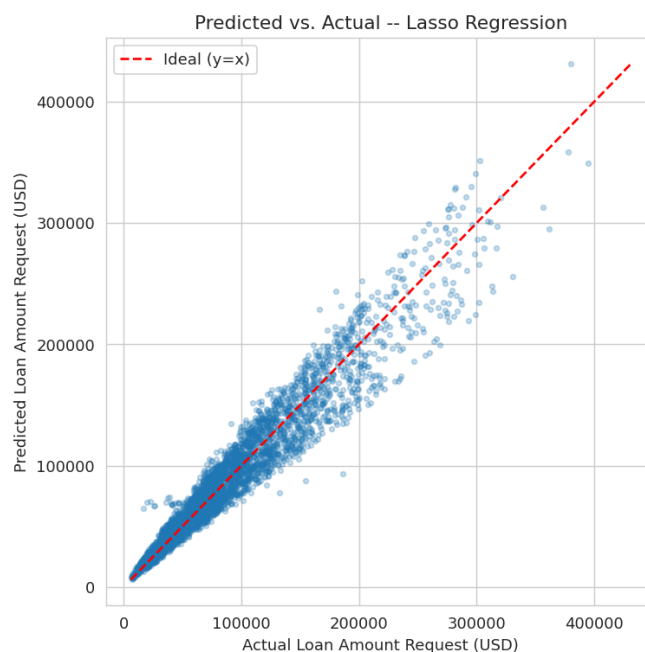


Figure 4: Predicted vs. actual Loan Amount Request – best model (Lasso Regression)

Inference: Points cluster tightly along the ideal $y = x$ line across almost the entire range, only fanning out slightly at the highest loan amounts – visually confirming the high test R^2 (≈ 0.936) and showing the model generalizes well.

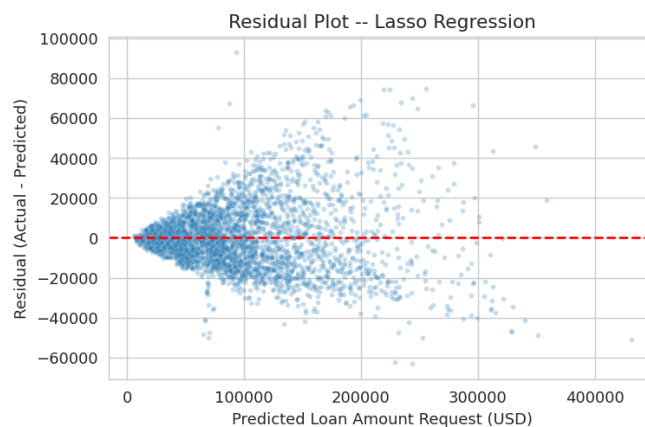


Figure 5: Residual plot – best model (Lasso Regression)

Inference: Residuals are centered close to zero (mean ≈ -309 , small relative to the target's scale) and show only mild fanning at higher predicted values – a much weaker heteroscedasticity pattern than the target's skew alone would suggest, since Property Price's near-linear relationship with the target absorbs most of that structure.

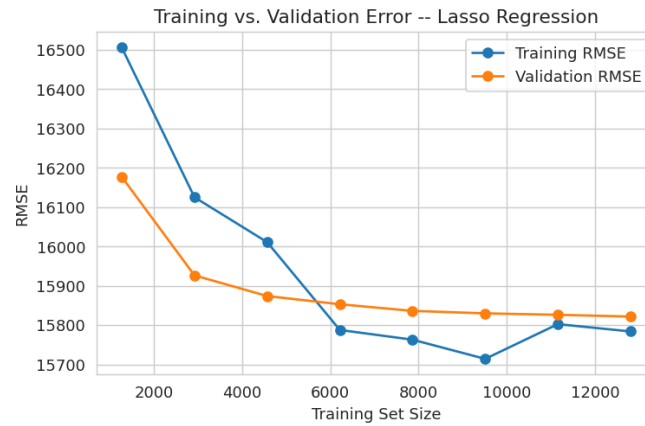


Figure 6: Learning curve (training vs. validation RMSE) – best model (Lasso Regression)

Inference: Training RMSE ($\approx 15,700$ – $16,500$) and validation RMSE ($\approx 15,800$ – $16,200$) stay close together at every training size and both decrease as more data is added, converging toward a shared, low-error plateau – the signature of a well-fit model with neither severe overfitting nor severe underfitting.

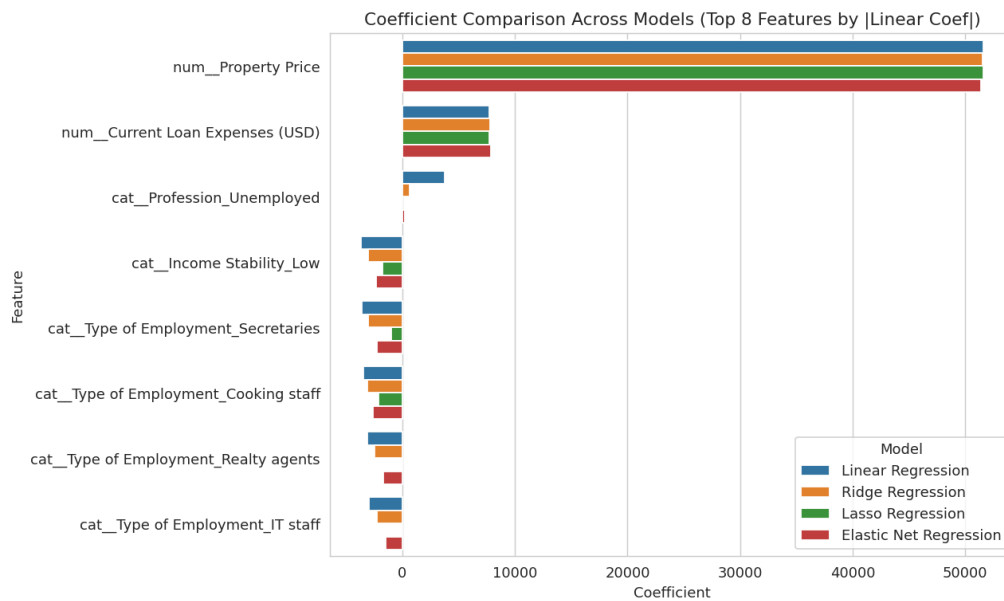


Figure 7: Feature importance – coefficient comparison across models (top 8 features by —Linear coefficient—)

Inference: Property Price dominates every model’s coefficients by an order of magnitude over any other feature (Linear: 51,583 vs. the next largest at 7,727 for Current Loan Expenses). Lasso zeroes out three of the remaining top-8 features entirely (Profession_Unemployed, Type of Employment_Realty agents, Type of Employment_IT staff), while Ridge and Elastic Net shrink but retain all of them – the expected Lasso-vs-Ridge sparsity contrast.

11 Discussion

Unlike a weaker-signal regression problem, this dataset – with Property Price as a feature – produces a strong, well-behaved fit: all four models achieve test $R^2 \approx 0.935$ – 0.936 . Because the unregularized Linear Regression baseline was already close to the regularized models’ performance, regularization strength was not the binding constraint here – there was very little overfitting for L_1/L_2 penalties to correct. Lasso achieved the best (marginally) test performance by trimming a handful of low-signal categorical dummy variables without disturbing the dominant numeric predictors. A limitation of this experiment is that with one feature (Property Price) so overwhelmingly dominant, the experiment offers limited insight into how these models would behave on a dataset with more diffuse, comparably-weighted feature signal; a useful follow-up would be to refit after removing Property Price to observe whether regularization becomes more consequential.

12 Conclusion

For the Predict Loan Amount Data dataset, Lasso Regression (tuned to $\alpha = 10$) is marginally the best-performing model on both cross-validation ($R^2 = 0.9303$) and the held-out test set (RMSE = 15,257.81, $R^2 = 0.9356$), though all four models are within 0.01% R^2 of one another. Property Price alone accounts for the overwhelming majority of predictive power. The learning curve shows converging, low training/validation error with no meaningful gap, indicating this experiment sits in a low-bias, low-variance regime where additional regularization tuning offers negligible practical benefit over plain Linear Regression.

13 References

- [1] Scikit-learn Developers, “Linear Models,” https://scikit-learn.org/stable/modules/linear_model.html
- [2] Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” https://scikit-learn.org/stable/modules/grid_search.html
- [3] Kaggle, “Predict Loan Amount Data,” <https://www.kaggle.com/datasets>